Source Code Documentation

UK Credit Portfolio Performance and Stress Analysis

Project technical documentation

31 October 2026

Scope: This project uses a synthetic portfolio. No customer, account, or lender is real. Results are illustrative and do not establish actual performance, arrears, losses, causality, or credit policy effectiveness.

Contents

1 Purpose and repository map 2

2 Data and analytical source flow 2

3 SQL files and table grain 3

4 Scenario implementation 3

5 Reproduction commands 4

6 Power BI implementation package 4

7 Quality records and supported claims 4

1 Purpose and repository map

This guide describes the source files that generate and validate the project outputs. The project follows small task stages: define the data contract, generate and validate a synthetic portfolio, calculate review metrics, add sourced UK context, create reporting artifacts, audit assisted commentary, and run a final evidence review.

Path	Role
`data_specification.md`, `data_dictionary.md`	Grain, keys, meanings, denominators, date rules, edge cases, and limitations.
`generation_rules.md`	Fixed seed and all synthetic generation assumptions.
`scripts/generate_portfolio.py`	Creates the account-month CSV with standard-library Python.
`scripts/validate_portfolio.py`	Checks schema, key, range, state, lifecycle, arithmetic, and monthly totals.
`sql/`	SQLite schema and monthly, risk-segment, vintage, delinquency, roll-rate, and reconciliation queries.
`scripts/run_sql_analysis.py`	Loads controlled data into SQLite, runs checks, and exports query results.
`scripts/build_review_workbook.py`	Writes the eight-sheet Excel review workbook from SQL exports.
`scripts/validate_review_workbook.py`	Checks workbook structure, source row counts, formulas, and cached ratios against SQL.
`scripts/build_scenarios.py`	Joins month-end Bank of England context and computes illustrative scenarios.
`scripts/validate_scenarios.py`	Checks exact date coverage, joins, assumptions, scenario calculations, and sensitivity rows.
`scripts/build_t05_reporting.py`	Embeds validated T03/T04 exports into the self-contained HTML dashboard preview.
`scripts/validate_t05_reporting.py`, `scripts/validate_t05_dashboard.js`	Check report lineage, KPI values, source coverage, and dashboard rendering/selection behavior.
`scripts/build_t06_commentary_inputs.py`	Produces 28 pre-calculated values, definitions, and source paths for the commentary audit.
`scripts/validate_t06_commentary.py`	Checks metric levels and movements against T03/T04, plus prompt, audit, and approved text controls.
`scripts/validate_t07_final.py`	Runs final stage validators and checks deliverable/evidence status.

2 Data and analytical source flow

The primary dataset is `data/synthetic_portfolio.csv`, with one account per month-end observation. It contains 55,145 rows for 2,000 generated accounts across 57 month ends, January 2022 through September 2026. Monetary source fields are generated to integer pence and serialized in GBP. The accepted file hash is `04dce168244871a60a66c62b9f538f86050d0e224b5bb7f9b62c8e6ae10f896e`.

1. `generate_portfolio.py` applies the documented fixed seed and writes the synthetic snapshots.

2. `validate_portfolio.py` compares the source rows with required lifecycle and arithmetic rules and writes monthly control totals.
3. `run_sql_analysis.py` loads the source/control totals, materializes `data/portfolio.sqlite`, runs eight reconciliations, and exports five analytical cuts.
4. `build_review_workbook.py` uses those SQL outputs and the fact file to create the review workbook. The workbook validator checks cached formula results against SQL.
5. `build_scenarios.py` exact-joins the three official Bank of England series to the portfolio by month-end, then calculates the explicit T04 sensitivities.
6. T05 embeds the accepted outputs for reporting. T06 assembles pre-calculated metrics, preserves its prompt and raw draft, and retains the human audit and approved commentary.

3 SQL files and table grain

- `schema.sql`: fact and monthly-control table definitions; financial values are integer pence in SQLite.
- `monthly_portfolio.sql`: monthly stocks, flows, rate numerators/denominators, utilisation, and simulated 30+ proxy rates.
- `risk_segment.sql`: monthly cuts by synthetic segment.
- `vintage_mob.sql`: origination cohort and months-on-book cuts, with observed and active populations distinguished.
- `delinquency_state.sql`: account and balance distribution across generated state buckets.
- `roll_rates.sql`: consecutive-month account transitions, including a separate CLOSED destination.
- `reconciliation_checks.sql`: data and aggregate consistency checks.

The T01 metric contract defines portfolio utilisation as summed active balance divided by summed active limit. Payment rate is active payment receipts divided by active amount due; final close-month receipts are reported separately. The generated delinquency state and representative DPD days are proxies because the generator has no dated bill or arrears ledger. Closing balance is not EAD or expected loss.

4 Scenario implementation

`data/boe_context_monthly.csv` contains Bank Rate (`IUMBEDR`), a representative quoted card lending rate (`IUMCCTL`), and an effective rate for interest-charging card balances (`CFMZ6IR`). Metadata, coverage, URLs, release timing, and access date are in `data/boe_source_register.csv`. The portfolio and external rates are separate populations.

The T04 scenario engine applies effective card-rate assumptions to the September 2026 synthetic closing balance, held constant for 12 months. All balance is assumed interest-bearing and immediately repriced. Bank Rate changes and card-rate shocks are independent; there is no estimated pass-through. Results are annualized mechanical cost proxies, not forecasts, customer costs, portfolio income, or loss projections.

5 Reproduction commands

Run from the repository root. The first group recreates the analysis. The final validator reruns the stage checks and records the outstanding native Power BI handoff.

```
python3 scripts/generate_portfolio.py
python3 scripts/validate_portfolio.py
python3 scripts/run_sql_analysis.py
python3 scripts/build_review_workbook.py
python3 scripts/validate_review_workbook.py
python3 scripts/build_scenarios.py
python3 scripts/validate_scenarios.py
python3 scripts/build_t05_reporting.py
python3 scripts/validate_t05_reporting.py
python3 scripts/build_t06_commentary_inputs.py
python3 scripts/validate_t06_commentary.py
python3 scripts/validate_t07_final.py
```

The generator horizon is frozen through September 2026. The saved Bank of England series are inputs to the reproducible analysis; updating them requires a new official-source review and updated metadata.

6 Power BI implementation package

The T05 folder contains a browser dashboard preview, Power BI import instructions, model relationships, and DAX measures. The summary model should use the controlled T03/T04 CSV outputs, not the full source snapshot file. The environment used for this build had no Power BI Desktop or PBIX authoring tool. No native PBIX or render test exists. A detailed, current step-by-step handoff is in `latex/src/powerbi_implementation_guide.tex` and its compiled PDF.

7 Quality records and supported claims

Task status is in `execution.md`; findings and dispositions are in `critic.md` and `update_log.md`. Detailed evidence includes T02 data validation, T03 SQL/workbook reconciliation, T04 scenario checks, T05 reporting checks, T06 commentary audit, and T07 final review.

Resume claims are explicitly deferred until overall project acceptance. A native Power BI report remains an open material deliverable; the current dashboard must be described as an interactive HTML preview plus a Power BI-ready package.